

User Hierarchy

Build instructions for the five-tier access model, job titles, and client scoping.

AUTHOR

Nikko Trinidad

AUDIENCE

Developers implementing the access layer

PAIRS WITH

docs/user-hierarchy-guide.pdf (audience: leadership and team)

What this spec gives you

This document is the build instruction for the access layer of the agency PM tool. It is the source of truth for what to ship. The leadership guide (user-hierarchy-guide.pdf) explains the model to humans. This spec turns the same model into typed code, a permission matrix, a data model, scoping rules, enforcement points, and an acceptance checklist.

Sections in order:

- **02 · System overview.** Three orthogonal axes and how they compose.
- **03 · Access vocabulary.** The two TypeScript unions (AccessRole, Action). Drop into `utils/access.ts`.
- **04 · Permission matrix.** The complete role-by-action lookup as code. Wraps with a `can()` helper.
- **05 · Data model.** Every entity that carries role, title, or assignment data.
- **06 · Client scoping.** `canSee()` rules for clients, cards, and members.
- **07 · Gate enforcement.** Where the gates fire: route guards, component gates, server-side rules.
- **08 · UI surfacing.** Where tier and title render in the interface.
- **09 · Seed and migration.** Map the current Operator tier onto the new model. Set Nikko as Owner.
- **10 · Acceptance criteria.** Per-tier walkthrough. Use as test cases.
- **11 · Implementation checklist.** Tickable steps in the order they should ship.

RULES OF ENGAGEMENT

When this spec disagrees with the leadership guide, this spec wins on data-model and gate-enforcement questions. The leadership guide wins on language and intent. If a rule isn't covered in either, raise it before inventing one; we will pick and add it to both docs.

Three axes, composed

Every member of the workspace carries three pieces of identity. They move independently and they compose at the gate.

Axis	Field	What it controls
Access tier	User.tier (AccessRole)	What actions the member is allowed to perform (manage workspace, approve time off, edit cards, view boards).
Job title	User.jobTitleId (string)	Identity. Drives the profile pill, the AM filter on Clients, and the cover-rule selector. Has no permission effect on its own.
Client assignment	Client.amId + Client.teamIds + Card.assigneeIds	Which clients and cards the member sees. Used by canSee() for Manager and Member tiers. Has no effect on Owner or Admin.

How they compose at the gate

A gate is the function call we make before doing anything interesting. The two-layer model:

utils/access.ts // compose pattern

```
function allowed(actor: User, action: Action, resource?: any) {
  // Layer 1: action-level gate (role vs action).
  if (!can(actor.tier, action)) return false;

  // Layer 2: resource scoping (only for the actions that
  // touch a specific client, card, or member).
  if (resource && !canSee(actor, resource)) return false;

  return true;
}
```

WHY TWO LAYERS

The action-level gate answers "is this kind of action ever allowed for this tier?" The scoping gate answers "is this specific resource visible to this actor?" Keeping them separate stops Action names from exploding (no edit:any-card vs edit:assigned-card) and lets the same scoping helper be reused across every action that touches a Client or a Card.

AccessRole and Action, as TypeScript

These two unions are the type-level foundation. Drop into **utils/access.ts**. The string values stay short and stable because they are used as Firestore-rule literals and as denormalised keys on the Workspace.memberRoles map.

AccessRole union

```
// utils/access.ts
// -----

// Five tiers, ordered by power. Owner sits at the top because
// they hold workspace ownership (transfer + delete). Admin is
// next. Manager and Member are scoped to assigned clients;
// Viewer is for external entities (clients, contractors)
// outside the organisation.
export type AccessRole =
  | 'owner'
  | 'admin'
  | 'manager'
  | 'member'
  | 'viewer';
```

Action is the union of every gated verb in the app. Naming convention: **verb:object**. Self-scoped actions (the actor's own resource) carry an *own* prefix in the object so the gate reads as a single intent without needing a runtime "is this mine" check.

Below: each group as a typed Action chunk. Concatenate them into a single union in **utils/access.ts**; they are split here only for readability.

Action union (part 1 of 2)

```

export type Action =
  // -- Workspace administration -----
  | 'manage:workspace'          // identity (name, initials, colour, logo)
  | 'manage:members'           // view list + invite + revoke + remove
  | 'manage:admins'            // assign/revoke Admin tier (Owner only)
  | 'manage:roles-below'       // assign Manager/Member/Viewer tier
  | 'manage:member-caps'       // edit capSoft / capMax
  | 'manage:employment'        // probationary / regular + regularizedAt
  | 'manage:job-titles'        // CRUD job-titles catalogue
  | 'manage:holidays'         // CRUD + sync + import/export + credit
  | 'manage:trash'             // view + restore + empty trash

  // -- Time off -----
  | 'submit:own-time-off'      // own request: submit, edit pending, cancel
  | 'approve:time-off'        // approve / deny anyone's request
  | 'manage:coverage-rules'    // edit coverage rules
  | 'view:team-balances'       // Ops -> Balances sub-tab
  | 'manage:leave-guidelines'  // edit guidelines doc
  | 'view:leave-guidelines'    // read-only viewer

```

Action union (part 2 of 2)

```

// -- Ops board -----
| 'nav:ops'           // see Ops in top navigation
| 'view:ops'          // view Ops board + capacity heatmap
| 'edit:ops'          // drag, assign, edit Ops tasks

// -- Clients and services -----
| 'view:clients'      // clients list + detail (scoped by canSee)
| 'create:client'     // add a new client
| 'edit:client'       // edit client header (industry, AM, site)
| 'archive:client'    // archive / delete a client
| 'manage:contacts'   // contacts + quick links
| 'edit:client-notes' // edit Notes tab
| 'create:service'    // add a service to a client
| 'edit:service'      // edit service settings
| 'archive:service'   // archive / delete a service

// -- Kanban boards (cards) -----
| 'view:boards'       // view a service kanban board
| 'create:card'        // create a new card
| 'assign:card'        // assign cards to other members
| 'edit:own-card'      // edit cards assigned to self
| 'edit:any-card'      // edit cards assigned to others
| 'comment:card'       // post a comment
| 'move:any-card'      // move any card between columns
| 'move:own-card'      // move own card between columns
| 'archive:card'       // archive a card

// -- Templates -----
| 'nav:templates'     // see Templates in top navigation
| 'view:templates'    // view templates list
| 'manage:templates'   // create, edit, archive templates
| 'reset:template'     // reset built-in template to default

// -- Other surfaces -----
| 'view:home'          // Home dashboard
| 'view:analytics'     // Analytics page
| 'view:weekly-wip'    // Weekly WIP meeting flow
| 'edit:weekly-wip'    // create captures + action items

// -- Workspace ownership -----
| 'transfer:workspace' // transfer ownership (Owner only)
| 'delete:workspace';  // delete the workspace (Owner only)

```

Role to action, as code

The matrix is the action-level gate. "Allowed" means the tier can perform the action when the resource (if any) is in scope. Resource scoping is enforced separately by `canSee()` in Section 06.

Encoded as a Record keyed by `AccessRole`. Each tier carries a Set of allowed Actions. The `can()` helper is a one-line lookup.

Permission bundles (part 1 of 2)

```
// utils/access.ts (continued)
// -----

const OWNER_ONLY: Action[] = [
  'transfer:workspace',
  'delete:workspace',
  'manage:admins',
];

const ADMIN_BASE: Action[] = [
  'manage:workspace', 'manage:members', 'manage:roles-below',
  'manage:member-caps', 'manage:employment',
  'manage:job-titles', 'manage:holidays', 'manage:trash',
  'approve:time-off', 'manage:coverage-rules',
  'view:team-balances', 'manage:leave-guidelines',
  'edit:ops', 'archive:client', 'archive:service',
  'edit:any-card', 'move:any-card', 'archive:card',
];

const MANAGER_BASE: Action[] = [
  'create:client', 'edit:client', 'manage:contacts',
  'edit:client-notes', 'create:service', 'edit:service',
  'create:card', 'assign:card', 'edit:own-card',
  'edit:any-card', 'move:any-card', 'archive:card',
  'view:ops', 'nav:ops',
];

const MEMBER_BASE: Action[] = [
  'edit:client-notes', 'edit:own-card', 'move:own-card',
];
```

Permission bundles + PERMISSIONS + `can()` (part 2 of 2)

```

const VIEWER_BASE: Action[] = [
  'view:weekly-wip', // read-only attendance
];

// Shared by every tier above Viewer:
const ALL_AUTHENTICATED: Action[] = [
  'submit:own-time-off', 'view:leave-guidelines',
  'view:home', 'view:analytics', 'view:weekly-wip',
  'edit:weekly-wip', 'view:clients', 'view:boards',
  'comment:card', 'nav:templates', 'view:templates',
  'manage:templates', 'reset:template',
];

export const PERMISSIONS: Record<AccessRole, Set<Action>> = {
  owner:   new Set([...ALL_AUTHENTICATED, ...ADMIN_BASE,
    ...MANAGER_BASE, ...OWNER_ONLY]),
  admin:   new Set([...ALL_AUTHENTICATED, ...ADMIN_BASE,
    ...MANAGER_BASE]),
  manager: new Set([...ALL_AUTHENTICATED, ...MANAGER_BASE]),
  member:  new Set([...ALL_AUTHENTICATED, ...MEMBER_BASE]),
  viewer:  new Set(VIEWER_BASE),
};

export function can(role: AccessRole, action: Action): boolean {
  return PERMISSIONS[role].has(action);
}

```

The full role-by-action lookup in human-readable form lives in Section 06 of the leadership guide (user-hierarchy-guide.pdf). Treat this code block as the binding form; if the two disagree, this code is the source of truth and the leadership guide gets updated.

Entities that carry role, title, or assignment data

Only the access-relevant fields are listed here. Existing fields stay untouched. Field types use TypeScript notation; if the back end uses Firestore, treat string types as document ID strings and Timestamp as the Firestore Timestamp type.

User

```
interface User {
  id: string;
  email: string;
  displayName: string;
  avatarUrl?: string;

  // Access tier (what the user is allowed to do).
  tier: AccessRole;

  // Job title (identity; references JobTitle.id).
  // Optional only for the Owner seed before catalogue is built.
  jobId?: string;

  // Employment status (drives leave entitlement + UI badges).
  employmentType: 'probationary' | 'regular';
  regularizedAt?: Timestamp;

  // Capacity caps (hours/week or card count, your choice;
  // capSoft is the planning target, capMax is the hard limit).
  capSoft?: number;
  capMax?: number;

  // Denormalised for fast lookups in server rules.
  workspaceId: string;
}
```

Workspace

```

interface Workspace {
  id: string;
  name: string;
  initials: string;
  colorHex: string;
  logoUrl?: string;

  // Exactly one Owner per workspace. Transfer rewrites this.
  ownerId: string;

  // Denormalised role map for server-side rule checks. Mirrors
  // User.tier for every member. Keep in sync via a Cloud
  // Function trigger on User writes.
  memberRoles: Record<string, AccessRole>; // userId -> tier

  createdAt: Timestamp;
}

```

Client

```

interface Client {
  id: string;
  workspaceId: string;
  name: string;
  industry?: string;
  website?: string;

  // Account Manager. References User.id. Must be a user whose
  // tier is 'manager' or higher.
  amId: string;

  // Additional assignees (Members and Managers who work on
  // this client). The AM is implicitly included; do not need
  // to also appear in teamIds.
  teamIds: string[];

  archived: boolean;
  archivedAt?: Timestamp;
}

```

JobTitle

```

interface JobTitle {
  id: string;
  workspaceId: string;
  name: string; // 'Account Manager', 'Designer', ...
  colorHex: string; // drives profile pill colour
  archived?: boolean; // soft-delete; preserves historical pills
}

```

Card (delta)

```
interface Card {
  // ... existing fields

  // Members who own the work on this card. Drives 'edit:own-card'
  // and 'move:own-card' for Member-tier users.
  assigneeIds: string[];
}
```

TimeOffRequest

```
interface TimeOffRequest {
  id: string;
  workspaceId: string;
  userId: string;           // the requester

  type: 'annual' | 'sick' | 'unpaid' | 'other';
  startDate: string;        // ISO date (yyyy-mm-dd)
  endDate: string;
  hours: number;
  reason?: string;

  status: 'pending' | 'approved' | 'denied' | 'cancelled';
  approverId?: string;      // a user with 'approve:time-off'
  decidedAt?: Timestamp;
  decisionNote?: string;

  createdAt: Timestamp;
}
```

Holiday + LeaveGuideline (brief)

Holiday and LeaveGuideline already exist on Phase 8. No schema changes needed for the access layer beyond confirming the gates: **manage:holidays** and **manage:leave-guidelines** are Owner + Admin; **view:leave-guidelines** is everyone-but-Viewer.

canSee() for clients, cards, and members

Owner and Admin see everything. Manager and Member see only the clients they are assigned to. Viewer sees only what leadership has explicitly shared (the Weekly WIP they are invited to, no default client access).

utils/scope.ts

```
// utils/scope.ts
// -----

export function canSeeClient(actor: User, client: Client): boolean {
  if (actor.tier === 'owner' || actor.tier === 'admin') return true;
  if (actor.tier === 'viewer') return false;
  // Manager or Member: must be the AM or in teamIds.
  return client.amId === actor.id
    || client.teamIds.includes(actor.id);
}

export function canSeeCard(actor: User, card: Card,
  client: Client): boolean {
  // Card visibility is always client-gated first.
  if (!canSeeClient(actor, client)) return false;
  // Owner / Admin / Manager see every card on a visible client.
  if (actor.tier !== 'member') return true;
  // Member: must be on the card's assigneeIds.
  return card.assigneeIds.includes(actor.id);
}

export function canSeeMember(actor: User, target: User): boolean {
  if (actor.tier === 'owner' || actor.tier === 'admin') return true;
  // Manager + Member can see themselves and members on the
  // same client(s). Viewer can see only themselves.
  if (actor.id === target.id) return true;
  if (actor.tier === 'viewer') return false;
  return shareAnyClient(actor.id, target.id);
}
```

ONE EDGE CASE WORTH FLAGGING

A Member assigned to a single card on a client gets the WHOLE client context (Overview, Notes, every service, every other card on those boards) by canSeeClient + canSeeCard composition. This is intentional and confirmed in the leadership brief. The Member needs the surrounding context to do their card well. The filter is the client, not the card.

Three layers, three places to fire

Every gate fires in three places. Skipping a layer is a security bug. They are cheap to add together; expensive to add later.

Layer A: Route guards

Wrap top-level routes so an unauthorised user is redirected before any panel renders. Templates is gated by **nav:templates**; Ops by **nav:ops**; Account Settings sub-pages by their respective gates.

Route guard pattern

```
// router.tsx
<Route path="/templates" element={
  <Guard action="nav:templates" fallback="/home">
    <TemplatesPage />
  </Guard>
} />

// components/Guard.tsx
export function Guard({ action, fallback, children }) {
  const actor = useCurrentUser();
  if (!can(actor.tier, action)) return <Navigate to={fallback} />;
  return <>{children}</>;
}
```

Layer B: Component gates

Inside a permitted route, individual buttons + sections gate by action. The `<Gate>` component hides the child if the actor cannot perform the action. Disabled-and-tooltipped is reserved for cases where the tier could do it but a scoping rule fails (so the user understands they would have the right on a different client).

Component gate pattern

```
// components/Gate.tsx
export function Gate({ action, resource, children, fallback = null }) {
  const actor = useCurrentUser();
  if (!can(actor.tier, action)) return fallback;
  if (resource && !canSee(actor, resource)) return fallback;
  return <>{children}</>;
}

// Usage in a client detail header:
<Gate action="edit:client" resource={client}>
  <button onClick={openEditModal}>Edit client</button>
</Gate>
```

Layer C: Server-side rules

Frontend gates can be bypassed. The server is the final answer. If using Firestore, write rules that read *Workspace.memberRoles* (denormalised) so each gate is one document read.

Firestore rule pattern

```
// firestore.rules (excerpt)

function actorRole(workspaceId) {
  return get(/databases/$(database)/documents/workspaces/$(workspaceId))
    .data.memberRoles[request.auth.uid];
}

function isOwnerOrAdmin(workspaceId) {
  let r = actorRole(workspaceId);
  return r == 'owner' || r == 'admin';
}

match /workspaces/{workspaceId}/clients/{clientId} {
  // view:clients (scoping enforced in code; rule only checks tier)
  allow read: if request.auth != null
    && actorRole(workspaceId) != null;

  // create:client (Owner, Admin, Manager)
  allow create: if isOwnerOrAdmin(workspaceId)
    || actorRole(workspaceId) == 'manager';

  // archive:client (Owner, Admin)
  allow delete: if isOwnerOrAdmin(workspaceId);
}
```

WHY DENORMALISE *memberRoles*

Firestore rules cannot do joins. Without the denormalised map on the *Workspace* document, every gate would require reading the *Member* subcollection. The map is updated by a Cloud Function trigger on *User* tier writes. Cost: one extra write per role change. Benefit: every read-gate becomes one extra document read instead of a query.

Where tier and title render

The two identity pieces show up in different places and read as different signals. Keep them distinct visually.

Surface	Shows	Why
Profile pill (avatar + chip on every page header)	Job title only	Identity. Tier is implied by what the user can do, not declared.
Member chip (Members list, card assignee)	Tier label + job title	Admin context: leadership needs both signals when managing the roster.
Account Settings -> Members	Tier dropdown + job title select	The place where Owner + Admin change either axis.
Client header AM filter	Job title 'Account Manager' filter	Filters by job title, not tier; lets a Manager-tier user with a different title (e.g. Strategist) be excluded from the AM dropdown.
Action visibility	Hidden when can() is false; disabled+tooltip when can() passes but canSee() fails	Distinguishes "not your tier" from "not your client" without surfacing the tier to the user.
Viewer empty states	Friendly "nothing shared with you yet" copy	Viewer accounts often land on a route with no data; the empty state should not read as a bug.

Map the current state onto the new model

The workspace already has members on the legacy tier vocabulary. The migration is mechanical:

Tier mapping

Legacy tier	New AccessRole	Notes
(none) -> Nikko Trinidad	owner	Exactly one Owner per workspace. Set Workspace.ownerId to Nikko's User.id.
Operations Manager	admin	Tier renamed; the old 'Operations Manager' label survives as a JobTitle entry, not a tier.
Account Manager	manager	Move to Manager tier. Set jobTitleId to the Account Manager JobTitle entry. AM-of-record on a Client uses User.id + jobTitle filter, not tier.
Operator (4 members today)	member	Bulk migrate. JobTitleId stays as the legacy Operator label until leadership reassigns.
Designer / Strategist / Manager (job titles, 0 members)	(create JobTitle, no User assignment)	Seed the JobTitle catalogue entries so the dropdown is ready when leadership hires into them.

Migration script outline


```

// scripts/migrate-access.ts
// Run once, idempotent. Wrap in a transaction per workspace.

const TIER_MAP: Record<string, AccessRole> = {
  'Operations Manager': 'admin',
  'Account Manager':    'manager',
  'Operator':           'member',
};

for (const workspace of allWorkspaces()) {
  // 1. Seed JobTitle catalogue (Account Manager, Designer,
  //    Strategist, Operator, Manager, Operations Manager).
  await seedJobTitles(workspace.id);

  // 2. Set Owner (Nikko by default; configurable per workspace).
  await setOwner(workspace.id, OWNER_USER_ID);

  // 3. For each member: map legacy tier -> new tier, attach
  //    jobTitleId by legacy-label match.
  for (const user of workspace.members) {
    const newTier = TIER_MAP[user.legacyTier] ?? 'member';
    const jt = await findJobTitleByName(workspace.id, user.legacyTier);
    await updateUser(user.id, {
      tier: newTier,
      jobTitleId: jt?.id,
      employmentType: user.employmentType ?? 'regular',
    });
  }

  // 4. Rebuild Workspace.memberRoles map from new tiers.
  await rebuildMemberRolesMap(workspace.id);
}

```

WHY THE OWNER IS A SEED, NOT A SETTING

There is exactly one Owner per workspace and it must exist before any other tier can be assigned (because the Owner is the only tier that can grant Admin).

Hard-code the seed Owner in the migration script; transfer to a different user is a v2 feature handled by transfer:workspace, not by editing a row.

Per-tier walkthroughs to run before sign-off

Each block below is a manual test plan for one tier. Sign in as a seed user at that tier and walk the surfaces in order. All assertions must hold or the access layer is not done.

Owner

- Account Settings is visible and every sub-page renders.
- Members tab shows the full roster + tier dropdowns include Admin (and Owner is read-only on self).
- Workspace identity (name, logo, colour) is editable.
- Transfer ownership + Delete workspace appear under Workspace -> Danger zone.
- Templates page is visible; can create + archive built-ins.
- Ops board is visible + every card is draggable.
- Clients list shows every client (no scoping).

Admin

- Account Settings is visible; the Admin tier appears in the Members dropdown for assignment to others.
- Cannot assign Admin to anyone else (manage:admins is Owner-only) -- option grayed out with tooltip.
- Workspace -> Danger zone shows only manage:trash; transfer + delete are hidden.
- Approve / deny time-off on Ops -> Time off is enabled.
- Holidays + Leave Guidelines are editable.
- Templates + Ops board behave the same as Owner.

Manager

- Top nav shows Home, Clients, Ops, Analytics, Weekly WIP, Templates.
- Account Settings shows only the My Account section (no Workspace block).
- Clients list is filtered to clients where `amId == self.id` OR `teamIds.includes(self.id)`.
- Create new client succeeds; the new client lists self as the AM by default.
- Cards on assigned clients are editable + movable; cards on unassigned clients are not visible.
- Time-off submit + view-balances works; approve does not.

Member

- Top nav shows Home, Clients, Analytics, Weekly WIP (NO Ops, NO Templates).
- Account Settings shows only the My Account section.
- Clients list is filtered to clients where the member is on amlId or teamIds OR assigned to any card on the client.
- On a visible client, every service + every card is viewable (full client context).
- Edit + move are limited to cards where assigneeIds.includes(self).
- Comments are allowed on any visible card (not just own).
- Time-off submit + view-balances works.

Viewer

- Top nav shows Weekly WIP only; no other pages are reachable by URL (route guards redirect).
- Account Settings shows Profile + Sign-in only.
- Clients, Templates, Ops, Analytics, Home all redirect with a 'You don't have access to this page' message.
- Leave Guidelines is hidden; time-off submission is hidden.
- Weekly WIP is read-only (no captures or action items can be created).
- Empty states read as deliberate, not broken.

Ship in this order

Tick each item as it ships. Order matters: the type layer lands first so every later step can reference it; the migration runs last because it depends on the schema.

Foundation

- Add `utils/access.ts` with `AccessRole` + `Action` unions (Section 03).
- Add `PERMISSIONS` const + `can()` helper (Section 04).
- Add `utils/scope.ts` with `canSeeClient`, `canSeeCard`, `canSeeMember` (Section 06).
- Add `useCurrentUser()` hook returning the actor (User shape from Section 05).

Data model

- Add `User.tier`, `User.jobTitleId`, `User.employmentType`, `User.regularizedAt`, `User.capSoft`, `User.capMax`.
- Add `Workspace.ownerId` + `Workspace.memberRoles`.
- Add `Client.amId` + `Client.teamIds`.
- Add `Card.assigneeIds`.
- Create `JobTitle` entity + Firestore (or DB) collection.
- Confirm `TimeOffRequest` + `Holiday` + `LeaveGuideline` exist; no schema changes needed.

Gates

- Add route guard (Section 07A).
- Add component gate (Section 07B).
- Wire route guards on `Templates`, `Ops`, every `Account Settings` sub-page.
- Wire component gates on every action button (create client, archive, etc.).
- Write Firestore rules using `actorRole(workspaceId)` helper (Section 07C).
- Add Cloud Function trigger to keep `Workspace.memberRoles` in sync with `User.tier` writes.

UI surfacing

- Profile pill shows job-title chip only (Section 08).
- Member chip + Members table show tier + job title.
- Client header AM filter reads from `JobTitle name == 'Account Manager'`.
- Action visibility: hide when `can()` fails; disable+tooltip when only `canSee()` fails.
- Viewer empty states across `Home`, `Clients`, `Templates`, `Ops`, `Analytics`.

Seed and migration

- Build scripts/migrate-access.ts using the outline in Section 09.
- Seed JobTitle catalogue (Account Manager, Designer, Strategist, Operator, Manager, Operations Manager).
- Set Nikko as Workspace.ownerId.
- Map legacy tier labels (Operations Manager -> admin, Account Manager -> manager, Operator -> member).
- Rebuild Workspace.memberRoles.
- Run on staging; spot-check each tier; then run on prod.

Verification

- Run the per-tier acceptance walkthroughs in Section 10.
- Confirm route-guard redirects work (try a Viewer URL on /templates).
- Confirm server rules reject API calls a tier shouldn't make (curl test or rules-emulator).
- Confirm Manager + Member only see assigned clients.
- Confirm Owner is the only tier with Transfer + Delete workspace buttons.
- Confirm the leadership guide matches what the build does; update either side if they disagree.

DEFINITION OF DONE

Every checkbox ticked, every acceptance walkthrough green, Firestore rules emulator passing on the rule fragments in Section 07C, and the migration run successfully on prod with Workspace.memberRoles populated for every workspace. Ship in that order.